

Agentic ADLC with GitHub Copilot

From JIRA Ticket to Merged PR — Knowledge, Specialised Agents & Spec-Driven Delivery

2 Days · Hands-on | **Advanced** | 8 modules · 14 labs + capstone | 40% theory · 60% demo + labs | Trainer: Rajesh Gheware

Day 1 — Knowledge, Context & Specialised Agents

Module 1 · The ADLC Operating Model & Token Economics

- Where the AI development lifecycle really differs from the SDLC
- The re-discovery tax:** what the agent re-learns every request, and its cost in AI credits, time and accuracy
- Baselining a ticket end to end — turns, tool calls, questions, rework, clock time
- Mode and **model selection per task:** mechanical, reasoning, review — when the cheap model is the right answer
- The context window as a budget: what belongs in context, in a file, or behind a tool call
- Interactive vs. delegated (asynchronous) agent — choosing per ticket

Module 2 · Knowledge Harnessing — the Repository as the Agent's Memory

- The layered model — **org** → **repository** → **path-scoped** → **task-scoped**
- Repository instructions, path-scoped instruction files, architecture notes, ADRs, conventions, runbooks as standing context
- Signal vs. bloat** — what to write down, what the agent infers, what actively misleads it
- Structuring for retrieval: scoping rules, indexes, module maps, navigable naming
- Keeping it current** — ownership, review cadence, staleness detection, knowledge updated in the same PR, CI checks that fail on drift
- What must never be written down: secrets, credentials, client data — and the leak paths

Module 3 · Prompt & Context Engineering for Accurate Output

- Anatomy of a high-yield instruction: goal · constraints · inputs · output contract · done criteria · stop conditions
- Skill files** — ad-hoc prompts turned into versioned, reviewed, parameterised assets under source control
- Grounding that suppresses hallucination: cite-the-file, quote-before-you-change, refuse-when-unknown, ask-don't-invent
- The failure taxonomy — invented APIs, silent scope drift, confidently wrong refactors, tests written to pass — and the pattern that catches each
- Context hygiene in a long session: what to compact, when to start clean
- Proving a prompt is better:** an eval set of representative tickets and a pass rate

Module 4 · Specialised Agents — Design, Coding, Test & Review

- Why one general agent underperforms four narrow ones: context pollution, conflicting goals, reviewer independence
- Defining an agent — role, goal, allowed tools, inputs/outputs, guardrails, hand-off contract, and **what it must never do**
- The four in practice: **Design** (options & trade-offs), **Coding** (inside the architecture), **Test** (proving requirements, not code), **Review** (adversarial, blind to the coder's rationale)
- Hand-off artifacts as the interface between agents: spec, plan, diff, test report, review report
- Orchestration — sequential hand-off, parallel drafts, review-before-merge gates, and where a human must sit
- Anti-patterns: agent sprawl, overlapping roles, definitions nobody maintains

Day 2 — Spec-Driven Delivery: JIRA → Code → PR → Confluence

Module 5 · Spec-Driven Development

- Why spec-first beats prompt-first: the spec is the durable artifact, the chat transcript is disposable
- The chain — **principles** → **specification** → **technical plan** → **task breakdown** → **implementation** — with a human gate between each
- Specs an agent can execute and a test can check: machine-verifiable acceptance criteria, explicit out-of-scope, stated assumptions
- Challenging the plan **before** any code exists
- Keeping spec, tests and code in lockstep; detecting spec drift at pull-request time
- When spec-driven development is overkill, and the lighter mechanic to use instead

Module 6 · The Ticket as the Unit of Work — JIRA over MCP MCP

- MCP in practice: what the agent can see, tool budgets, and why *pulling* a ticket beats pasting one
- JIRA hygiene that makes a ticket agent-ready** — acceptance criteria, definition of ready, linked context, sizing to what an agent can finish
- Ticket → spec → plan → tasks, with the ticket key as the traceability anchor through branch, commit, PR and page
- Reading the surroundings: epics, linked issues, prior art, the design decision made three sprints ago
- Under-specified tickets — making the agent interrogate and clarify rather than invent
- Write-back guardrails:** what the agent may change unattended, what needs a human, how the audit trail is kept

Module 7 · Multi-Repo Engineering with Agents

- Why agents degrade across repositories: no shared context, no cross-repo search, contract changes landing out of order
- Workspace strategies — multi-root workspaces, per-repo instructions, a shared knowledge repository as common ground
- Contract-first cross-repo change:** API/schema first, producer and consumer sequenced, compatibility window, no big-bang merge
- Coordinated pull requests — ordering, linking, what merges first, and the rollback story when only half of it lands
- Keeping cross-repo context cheap: indexes, interface summaries and generated clients instead of full checkouts in context
- Shared libraries and versioning — what an agent may change, what needs a human owner

Module 8 · Closing the Loop — Review, Pull Requests & Documentation

- MCP
- The review agent on a real change set — what it reliably catches, its blind spots, and why the human still signs
- Reviewing **other people's AI-written code:** the defect classes that survive a confident PR description
- Pull requests the agent is genuinely good at — description tied to the spec, test evidence, risk & rollback notes
- Confluence write-back over MCP** — decisions, contracts and runbooks are worth generating; narrated diffs are not
- Documentation that feeds the next ticket — the loop closes back to Module 2
- Rollout in a team: standards, ownership of agent and skill files, measuring whether any of it worked
- Capstone** — one ticket end to end: pull → spec → plan → implement across both repos → test → review → PR → Confluence → knowledge-base update

Scope note — SRE / operations agents are deliberately excluded. CI/CD pipelines are already owned by the teams and the surrounding tooling is team-specific, so the course stops at the merged pull request and the updated documentation.